



TP 4 — Méthodes de segmentation

Imports nécessaires

Les librairies usuelles :

- pandas, numpy
- matplotlib, seaborn
- scipy.cluster.hierarchy, scipy.spatial.distance
- sklearn.preprocessing, sklearn.cluster
- gower pour la distance de Gower

Objectif

Le premier objectif consiste à utiliser la classification ascendante hiérarchique pour produire des groupes homogènes de fromages.

Le deuxième objectif consiste à identifier des groupes homogènes de véhicules, partageant des caractéristiques similaires.

Nous utiliserons essentiellement une approche à base des centres mobiles (k-means).

Enfin, comparer les résultats obtenus et penser à des méthodes mixtes en se basant sur CAH, K-means et ACP.

Données

- **cars.xls** : ce fichier de données est composé de 38 véhicules décrits par une série de variables. La source des données est un magazine américain. L'objectif est de produire des groupes homogènes de véhicules.
- **fromage.txt** : ce fichier représente un ensemble de fromages (29 observations) décrits par leurs propriétés nutritives (ex. protéines, lipides, etc. ; 9 variables).
- **Les librairies usuelles :**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage, cut_tree
from scipy.spatial.distance import pdist
import gower # pip install gower
```

EXERCICE 1 — K-means sur les données "cars.xls"

1. Charger les données

```
DF_cars = pd.read_excel("cars.xls")
```

2. Résumé statistique

```
print(DF_cars.describe(include='all'))
```

3. Matrice de dispersion (pairs plot)

```
sns.pairplot(DF_cars)
```

```
plt.show()
```

4. Distance de Gower

Utile pour données mixtes (quantitatives + qualitatives).

```
dis_cars = gower.gower_matrix(DF_cars)
```

Le TP R utilise ensuite **cr.cars**, qui n'est pas défini explicitement.

En Python, comme Gower ne fournit pas directement un espace métrique vectoriel, on fera du k-means **seulement sur les variables quantitatives**, ce qui est l'usage normal du k-means.

```
# Extraction des colonnes quantitatives
```

```
DF_quant = DF_cars.select_dtypes(include=np.number)
```

```
# centrage-réduction
```

```
scaler = StandardScaler()
```

```
cr_cars = scaler.fit_transform(DF_quant)
```

5. Segmentation k-means (3 classes)

```
kmeans_3 = KMeans(n_clusters=3, n_init=5).fit(cr_cars)
```

6. Afficher les résultats

```
print("Inertie :", kmeans_3.inertia_)
```

```
print("Centres :", kmeans_3.cluster_centers_)
```

7. Afficher les clusters

```
print(kmeans_3.labels_)
```

```
print(pd.Series(kmeans_3.labels_).value_counts())
```

8. Choix du nombre de clusters — méthode du coude

Equivalent de l'inertie expliquée.

```
inertie_expl = []
```

```
for k in range(1, 11):
```

```
    km = KMeans(n_clusters=k, n_init=5).fit(cr_cars)
```

```
    totss = km.inertia_
```

```
    inertie_expl.append(totss)
```

```
plt.plot(range(1, 11), inertie_expl, marker='o')
```

```
plt.xlabel("Nombre de groupes")
```

```
plt.ylabel("Inertie intra-classe")
```

```
plt.title("Méthode du coude")
```

```
plt.show()
```

Remarque : en Python, `inertia_` = somme des distances intra-classes.

Pour obtenir une version comparable R (% inertie expliquée) :

```
inertie_expl2 = []
```

```
totss = inertie_expl[0]
```

```
for k in range(2, 11):
```

```
    inertie_expl2.append(1 - inertie_expl[k-1] / totss)
```

```
plt.plot(range(2, 11), inertie_expl2, marker='o')
```

```
plt.xlabel("Nombre de groupes")
```

```
plt.ylabel("Proportion d'inertie expliquée")
```

```
plt.title("Méthode du coude (inertie expliquée)")
```

```
plt.show()
```

9. Segmentation finale

Choisir le nombre de classes déterminé par le coude :

```
nbrClasse = 3
```

```
kmeans_final = KMeans(n_clusters=nbrClasse, n_init=5).fit(cr_cars)
```

10. Visualisation (Drive_Ratio vs Weight)

```
plt.scatter(DF_cars["Drive_Ratio"], DF_cars["Weight"], c=kmeans_final.labels_)
```

```
plt.xlabel("Drive Ratio")
```

```
plt.ylabel("Weight")
```

```
plt.title("Clusters K-means")
```

```
plt.show()
```

Questions supplémentaires :

Q1.

Après avoir calculé la distance de Gower, utiliser une méthode de clustering adaptée aux distances mixtes

(ex : **k-medoids** / **PAM**) et comparer la partition obtenue avec celle du k-means.

- Réaliser un clustering PAM
 - Comparer les groupes via une matrice de confusion
 - Interpréter les différences
-

Q2.

Standardiser uniquement les variables quantitatives puis réexécuter K-means.

Comparer les clusters aux clusters obtenus avec les données non standardisées.

Q3.

Réaliser une **ACP** (Analyse en composantes principales) sur les variables quantitatives.

Projeter les individus sur le plan factoriel et superposer les clusters obtenus par k-means.

- Permet de visualiser la structuration des groupes
 - Identifier quelles variables séparent les classes
-

Q4.

Tester plusieurs valeurs de nstart dans K-means (1, 5, 20, 50).
Observer l'effet sur :

- l'inertie finale
 - la stabilité des clusters obtenus
 - le temps de calcul
-

Q5.

Utiliser d'autres indices de validation du partitionnement :

- Silhouette
- Calinski-Harabasz
- Davies-Bouldin

Comparer ces indices pour $k = 2$ à 10.

Q6.

Calculer la **distance moyenne intra-cluster** et **inter-cluster** pour la solution finale.
Commenter la qualité de séparation des groupes.

Q7.

Pour chaque cluster, calculer les statistiques descriptives (moyennes, médianes, écart-types) des variables.
Interpréter les profils caractéristiques de chaque groupe de véhicules.

=====

EXERCICE 2 — CAH sur les données "fromage.txt"

=====

1. Charger les données

```
fromage = pd.read_csv("fromage.txt", sep="\t", decimal=".", index_col=0)
```

2. Résumé statistique

```
print(fromage.describe())
```

3. Graphe de corrélation

```
sns.pairplot(fromage)
```

```
plt.show()
```

4. Centrage-réduction

```
scaler = StandardScaler()
```

```
fromage_cr = scaler.fit_transform(fromage)
```

5. Matrice de distances

```
fromage_dist = pdist(fromage_cr, metric='euclidean')
```

6. Dendrogramme (méthode de Ward)

```
cah_ward = linkage(fromage_dist, method='ward')
```

7. Affichage

```
plt.figure(figsize=(10, 5))
```

```
dendrogram(cah_ward, labels=fromage.index)
```

```
plt.title("Dendrogramme - Ward")
```

```
plt.show()
```

8. Analyse des sauts d'inertie

a) Les hauteurs (inerties)

```
inertie = np.sort(cah_ward[:, 2])[:-1]
```

```
plt.plot(inertie, drawstyle='steps-post')
```

```
plt.xlabel("Nombre de classes")
```

```
plt.ylabel("Inertie")
```

```
plt.show()
```

b) Zoom sur les 15 premiers sauts

```
plt.plot(inertie[:15], drawstyle='steps-post')
```

```
plt.xlabel("Nombre de classes")
```

```
plt.ylabel("Inertie")
```

```
plt.show()
```

c) Points de rupture

```
plt.plot(inertie[:15], drawstyle='steps-post')
```

```
plt.scatter([2,4,5], inertie[[2,4,5]], color=['green','red','blue'], s=80)
```

```
plt.show()
```

d) Visualisation des partitions (équivalent de rect.hclust)

Il n'existe pas d'équivalent direct, mais on peut découper l'arbre :

```
groupes_2 = cut_tree(cah_ward, n_clusters=2).flatten()
```

```
groupes_4 = cut_tree(cah_ward, n_clusters=4).flatten()
```

```
groupes_5 = cut_tree(cah_ward, n_clusters=5).flatten()
```

e) Découpage final en 4 classes

```
groupes_cah = cut_tree(cah_ward, n_clusters=4).flatten()
```

9. Lister les groupes

```
res = pd.DataFrame({"Fromage": fromage.index, "Groupe": groupes_cah})
```

```
print(res.sort_values("Groupe"))
```

Questions supplémentaires :

Q1.

Comparer le dendrogramme de Ward à ceux obtenus avec :

- single linkage
- complete linkage
- average linkage

Quels changements observe-t-on dans la structure hiérarchique ?

Q2.

Calculer l'indice de cophenetic correlation (corrélation cophénétique) pour évaluer la qualité de l'arbre hiérarchique (Ward). Comparer cet indice avec les autres méthodes d'agrégation.

Q3.

Projeter les données standardisées dans un espace ACP.
Représenter les clusters proposés (k=4) sur le plan factoriel.
Comment les groupes se séparent-ils dans l'espace factoriel ?

Q4.

Pour les partitions en 2, 3, 4 et 5 groupes, analyser les **profils moyens** des fromages sur les variables nutritives :

- Moyenne par groupe
 - Valeurs minimales/maximales
 - Variables les plus discriminantes
-

Q5.

Comparer les regroupements issus de la CAH avec un clustering par K-means ($k=4$).
Produire une matrice de contingence des deux partitions.
La CAH et le K-means proposent-ils une organisation similaire ?

Q6.

Utiliser la méthode du **gap statistic** ou de la **silhouette** pour estimer le nombre optimal de classes.
Comparer ce résultat aux sauts d'inertie du dendrogramme.

Q7.

Supprimer l'un des fromages considéré comme atypique (valeurs extrêmes),
refaire la CAH et observer l'impact sur :

- La structure du dendrogramme
- Les sauts d'inertie
- Les clusters finaux